Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 3

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Персистентность и изоляция

Обучающийся: Лясин Илья Игоревич

Факультет: 11.03.02 Прикладная информатика

Группа: K3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Лясин И.И.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Стр.

СОДЕРЖАНИЕ **2**

ВВЕДЕНИЕ **3**

1 Основная часть **4**

1.1 Этап 1. Запуск контейнера с именованным volume 4

1.2 Этап 2. Тест персистентности: пересоздание контейнера 5

1.3 Этап 3. Bind mount для конфигурации 6

1.4 Этап 4. Пользовательские сети для изоляции 7

1.5 Этап 5. Финальный артефакт 8

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Цель работы: Подключаем к приложению PostgreSQL, учимся хранить данные вне контейнера и разбираемся с сетями Docker.

В рамках работы рассматриваются основные механизмы контейнеризации: именованные тома для хранения данных, bind mount для передачи конфигурации, а также пользовательские сети для изоляции и связывания контейнеров. Практическая часть выполняется с использованием персональных данных, приведенных в таблице 1.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	466570
Персональный порт (app)	5500
Персональный порт (assistant)	8070
Персональный префикс	lyasin
Секретный ключ	secret70lyasin
Имя базы данных	lyasin70

1 Основная часть

1.1 Этап 1. Запуск контейнера с именованным volume

В ранее склонированном репозитории перейдём в ветку `lab-2` и создадим новую `lab-3`. Далее создадим новый том.

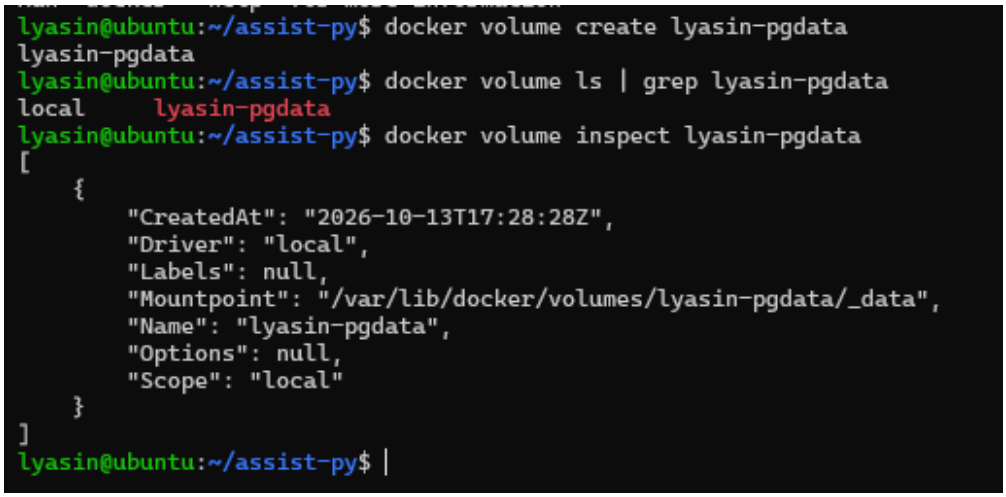


Рисунок 1.1 — Создание нового тома

И запустим postgres с созданным томом (рисунок 1.2). Том будет подключаться как директория `var/lib/postgresql/data` внутри контейнера. Именно эта директория содержит базы данных в postgres.



Рисунок 1.2 — Запуск postgres с созданным томом

Запустим контейнер с приложением ассистент (рисунок 1.3). Мы передаем переменную `DATABASE_URL` как адрес `host.docker.internal` - DNS имя хоста. Т.е. приложение получает доступ к БД через сеть хоста, также как пользователь подключается к БД через localhost.

```
lyasin@ubuntu:~/assist-py$ docker run -d \
  --name lyasin-assistant \
  --add-host=host.docker.internal:host-gateway \
  -p 8070:5500 \
  -e APP_OWNER=lyasin \
  -e APP_SECRET=secret70lyasin \
  -e DATABASE_URL="postgresql://user_70:pgpass_70@host.docker.internal:5500/assistant_70" \
  lyasin-assistant:v3
0eb0ff4d52569802862e18d658266c5d72a436767cef07179455b5fd0f782288
lyasin@ubuntu:~/assist-py$
```

Рисунок 1.3 — Запуск контейнера с приложением ассистент

И проверим его работоспособность (рисунок 1.4).

```
lyasin@ubuntu:~/assist-py$ curl http://localhost:8070/
{"message":"👋 Привет, lyasin!","service":"personal-assistant","version":"2.0","endpoints":["/me","/notes","/weather/{city}","/secret/{key}"]}lyasin@ubuntu:~/assist-py$
lyasin@ubuntu:~/assist-py$ curl http://localhost:8070/health
{"status":"healthy","owner":"lyasin"}lyasin@ubuntu:~/assist-py$
lyasin@ubuntu:~/assist-py$ curl -X POST http://localhost:8070/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР3: данные в БД"}'
{"status":"saved","note_id":1,"owner":"lyasin","text":"ЛР3: данные в БД"}lyasin@ubuntu:~/assist-py$
lyasin@ubuntu:~/assist-py$ curl http://localhost:8070/notes
{"owner":"lyasin","count":1,"notes":[{"id":1,"text":"ЛР3: данные в БД","owner":"lyasin","created_at":"2026-10-13T18:28:15.046539"}]}lyasin@ubuntu:~/assist-py$
```

Рисунок 1.4 — Проверка работоспособности приложения

1.2 Этап 2. Тест персистентности: пересоздание контейнера

Проверим, что приложение сохраняет информацию после перезапуска контейнера (рисунок 1.5).

```
lyasin@ubuntu:~/assist-py$ docker stop lyasin-assistant
lyasin-assistant
lyasin@ubuntu:~/assist-py$ docker rm lyasin assistant
Error response from daemon: No such container: lyasin
Error response from daemon: No such container: assistant
lyasin@ubuntu:~/assist-py$ docker rm lyasin-assistant
lyasin-assistant
lyasin@ubuntu:~/assist-py$ docker run -d \
  --name lyasin-assistant \
  --add-host=host.docker.internal:host-gateway \
  -p 8070:5500 \
  -e APP_OWNER=lyasin \
  -e APP_SECRET=secret70lyasin \
  -e DATABASE_URL="postgresql://user_70:pgpass_70@host.docker.internal:5500/assistant_70" \
  lyasin-assistant:v3
bb36fe1f7d2f785771fa7b0d79d1ef26412442491214e27b78abd6e18809b19d
lyasin@ubuntu:~/assist-py$
```

Рисунок 1.5 — Проверка сохранения информации после перезапуска

Выполним запрос к приложению (рисунок 1.6).

```
lyasin@ubuntu:~/assist-py$ curl http://localhost:8070/notes
{"owner":"lyasin","count":1,"notes":[{"id":1,"text":"ЛР3: данные в БД","owner":"lyasin","created_at":"2026-10-13T18:28:15.046539"}]}
```

Рисунок 1.6 — Запрос к приложению

Артефакт персистентности (рисунок 1.7).

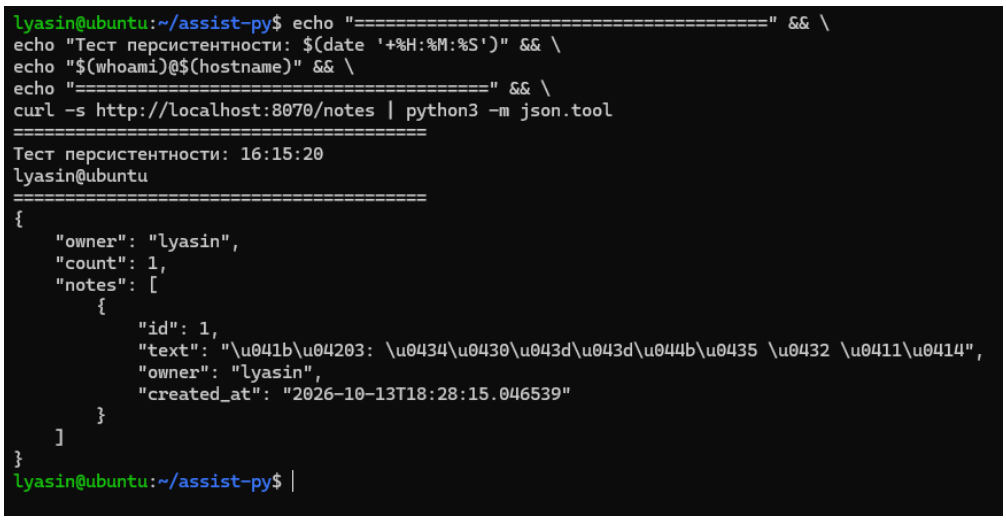


Рисунок 1.7 — Артефакт персистентности

1.3 Этап 3. Bind mount для конфигурации

Изменим конфигурационный файл `config/app.conf` (рисунок 1.8).

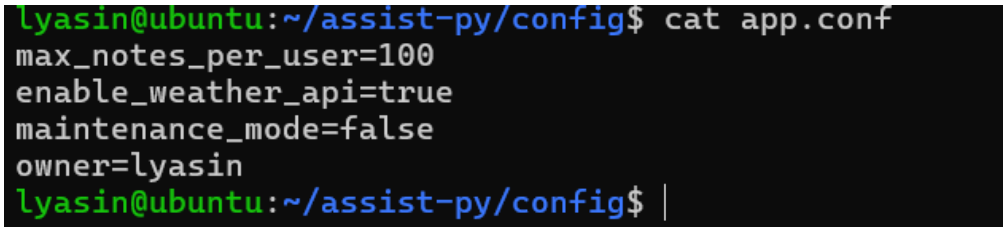


Рисунок 1.8 — Изменение конфигурационного файла

Теперь запустим новый контейнер, примонтировав конфигурацию как `bind mount`. Затем зайдём внутрь контейнера и проверим, что конфигурация действительно доступна изнутри контейнера (рисунок 1.9). Конфиг подключен как read-only, т.е. конфигурация производится только извне контейнера.

```
lyasin@ubuntu:~/assist-py$ docker run -d \
--name lyasin-assistant-config \
-p 8070:8070 \
-e APP_OWNER=lyasin \
-e APP_SECRET=secret70lyasin \
--add-host=host.docker.internal:host-gateway \
-e DATABASE_URL="postgresql://user_70:pgpass_70@host.docker.internal:5500/assistant_70" \
-v $(pwd)/config:/app/config:ro \
lyasin-assistant:v3
1e8f72d97893473e61ab4ff8929bfad6e4b155bea6c15b37db5534acc189b2cf
lyasin@ubuntu:~/assist-py$ docker exec -it lyasin-assistant-config sh
# cat /app/config/app.conf
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=false
owner=lyasin
# exot
sh: 2: exot: not found
# exit
lyasin@ubuntu:~/assist-py$
```

Рисунок 1.9 — Проверка доступности конфигурации внутри контейнера

1.4 Этап 4. Пользовательские сети для изоляции

Создадим пользовательскую сеть и удалим ненужные контейнеры с прошлых этапов (рисунок 1.10).

```
lyasin@ubuntu:~/assist-py$ docker network create lyasin-backend
3de912d9515f8ec612ac669dd7db48db7fa840bb77b95a233077087a6f7b0828
lyasin@ubuntu:~/assist-py$ docker stop lyasin-postgres lyasin-assistant-config
docker rm lyasin-postgres lyasin-assistant-config
lyasin-postgres
lyasin-assistant-config
lyasin-postgres
lyasin-assistant-config
lyasin@ubuntu:~/assist-py$
```

Рисунок 1.10 — Создание пользовательской сети и удаление контейнеров

Запустим БД и приложения в новой сети (рисунки 1.11 и 1.12).

```
lyasin@ubuntu:~/assist-py$ docker run -d \
  --name lyasin-postgres \
  --network lyasin-backend \
  -e POSTGRES_USER=user_70 \
  -e POSTGRES_PASSWORD=pgpass_70 \
  -e POSTGRES_DB=assistant_70 \
  -v lyasin-pgdata:/var/lib/postgresql/data \
  postgres:15
a531cad5df4f026472da7ac1145188905d7042d7c5d639d09723d6dc8af48929
```

Рисунок 1.11 — Запуск БД в новой сети

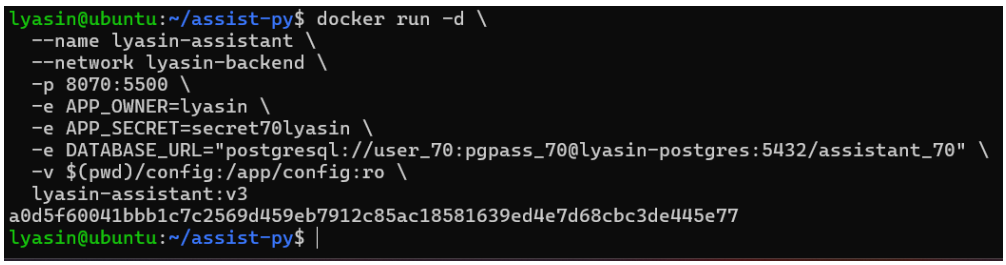


Рисунок 1.12 — Запуск приложения в новой сети

Благодаря тому, что контейнеры запущены в одной сети, мы можем прописать адрес БД, используя имя контейнера, так как для пользовательских сетей запускается docker DNS, в отличии от default.

Проверим доступность приложения (рисунок 1.13).

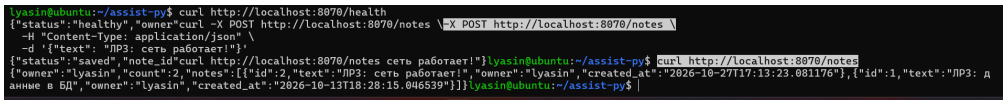


Рисунок 1.13 — Проверка доступности приложения

Докажем на практике, что контейнеры изолированы, если находятся в разных сетях, и доступны по имени, если находятся в одной сети (рисунок 1.14).

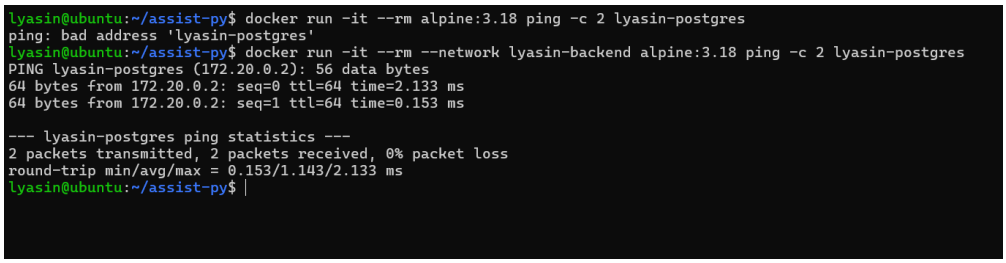


Рисунок 1.14 — Демонстрация изоляции и доступности по имени

1.5 Этап 5. Финальный артефакт

Выведем финальный артефакт лабораторной работы (рисунок 1.15).

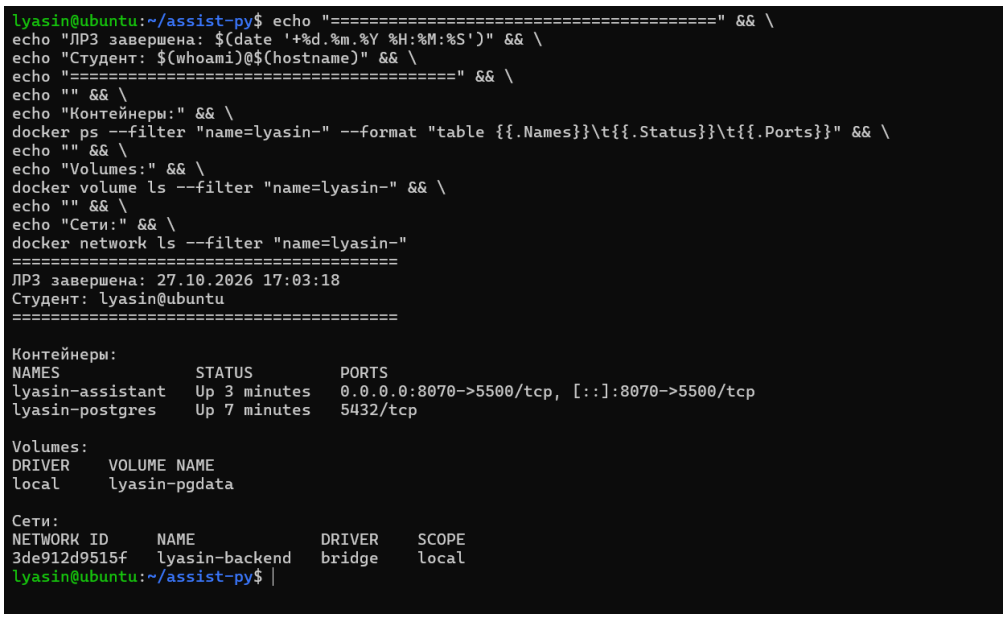


Рисунок 1.15 — Финальный артефакт лабораторной работы

На рисунке 1.15 видны:

1. оба контейнера (`lyasin-postgres`, `lyasin-assistant`);
2. именованный volume (`lyasin-pgdata`);
3. пользовательская сеть (`lyasin-backend`).

ЗАКЛЮЧЕНИЕ

В ходе практической работы была изучена экосистема Docker: освоены базовые команды, создание и использование именованных volume, bind mount для конфигурации, а также пользовательские сети для изоляции контейнеров. Продемонстрирована персистентность данных при пересоздании контейнера и работа docker DNS в пользовательской сети.